High Tech Marketing Text Analysis

Brand: Motorola Razr 2025

Group 12?

Introduction

Explain shit here philippe 8=====D

Sample Selection

Data Sources

Explain which sources we used to grab the data and why these are good sources for the analysis

Sampling representation & Bias

Explain if the sample is representative of all customers, and how platform choice may skew results e.g Reddit chuds are not normal people.

Data Collection

```
fetch_reddit_thread <- function(thread_url, limit = 500, attempts = 5) {
  api_url <- paste0(thread_url, ".json?limit=", limit)
  response <- RETRY(
    "GET",
    api_url,
    user_agent("Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) C
    times = attempts,
    pause_base = 2,
    pause_cap = 20,
    terminate_on = c(400, 401, 403, 404)
  )

  if (http_error(response)) {
    stop("HTTP ", status_code(response), " when fetching ", api_url)
  }

  payload <- fromJSON(content(response, as = "text", encoding = "UTF-8"), simplifyVector = FALSE)
  post <- payload[[1]]$data$children[[1]]$data
```

```r
flatten_comments <- function(children) {
  if (!is.list(children)) {
    return(data.frame())
  }
  rows <- list()

  for (child in children) {
    if (!is.null(child$kind) && child$kind == "t1") {
      comment <- child$data
      # sanitize body to avoid embedded newlines/commas breaking CSVs
      body_text <- comment$body %||% comment$body %||% NA_character_
      if (!is.null(body_text)) body_text <- gsub("[\r\n]+", " ", body_text)

      rows[[length(rows) + 1]] <- data.frame(
        type = "comment",
        id = comment$id %||% NA_character_,
        parent_id = comment$parent_id %||% NA_character_,
        author = comment$author %||% NA_character_,
        body = body_text,
        score = comment$score %||% NA_real_,
        created_utc = comment$created_utc %||% NA_real_,
        stringsAsFactors = FALSE
      )
      replies <- if (is.list(comment$replies) && !is.null(comment$replies$data$children)) {
        comment$replies$data$children
      } else {
        list()
      }
      if (length(replies) > 0) {
        rows <- c(rows, flatten_comments(replies))
      }
    }
  }

  if (length(rows) == 0) {
    return(data.frame())
  }

  # keep only well-formed data.frames
  rows <- Filter(function(x) is.data.frame(x) && ncol(x) > 0, rows)
  if (length(rows) == 0) return(data.frame())

  tryCatch(
    dplyr::bind_rows(rows),
    error = function(e) {
      message("Failed to bind comment rows: ", conditionMessage(e))
      # as a last resort, coerce each row to a 1-row data.frame with the expected columns
      cols <- c("type","id","parent_id","author","body","score","created_utc")
      repaired <- lapply(rows, function(r) {
        if (!is.data.frame(r)) return(NULL)
        missing <- setdiff(cols, names(r))
        for (m in missing) r[[m]] <- NA
        r[cols]
```

```
        })
        repaired <- Filter(Negate(is.null), repaired)
        if (length(repaired) == 0) return(data.frame())
        dplyr::bind_rows(repaired)
      }
    )
  }

  comments <- flatten_comments(payload[[2]]$data$children %||% list())

  # sanitize post body/title
  post_body <- post$selftext %||% post$title %||% NA_character_
  if (!is.null(post_body)) post_body <- gsub("[\r\n]+", " ", post_body)

  post_df <- data.frame(
    type = "post",
    id = post$id %||% NA_character_,
    parent_id = NA_character_,
    author = post$author %||% NA_character_,
    body = post_body,
    score = post$score %||% NA_real_,
    created_utc = post$created_utc %||% NA_real_,
    stringsAsFactors = FALSE
  )

  if (!is.data.frame(comments)) comments <- data.frame()
  tryCatch(
    dplyr::bind_rows(post_df, comments),
    error = function(e) {
      message("Failed to combine post and comments: ", conditionMessage(e))
      # return post only as fallback
      post_df
    }
  )
}

# if the cache exists and has content, use it; otherwise fetch fresh Reddit data.
suppressWarnings({
  reddit_cache <- "data/reddit_content.csv"
  cache_ready <- file.exists(reddit_cache) && file.info(reddit_cache)$size > 1

  if (cache_ready) {
    raw_df <- tryCatch(
      read.csv(reddit_cache, stringsAsFactors = FALSE),
      error = function(e) {
        message("Reddit cache could not be read: ", conditionMessage(e))
        data.frame()
      }
    )

    if (nrow(raw_df) > 0) {
      cat("Loaded Reddit content from cache:", nrow(raw_df), "\n")
    } else {
```

```
      message("Reddit cache is empty, fetching fresh data instead.")
      cache_ready <- FALSE
    }
  }

  if (!cache_ready) {
    # try to search more pages if the installed RedditExtractoR supports it
    reddit_search_pages <- 20
    reddit_max_threads <- 50

    reddit_posts <- tryCatch({
      args <- list(keywords = "motorola razr 2025", subreddit = "Android", sort_by = "top", page_thresh
      do.call(find_thread_urls, args)
    }, error = function(e) {
      # fallback if the package version doesn't accept page_threshold
      tryCatch(
        find_thread_urls(keywords = "motorola razr 2025", subreddit = "Android", sort_by = "top"),
        error = function(e2) {
          message("Reddit search failed: ", conditionMessage(e2))
          data.frame(url = character())
        }
      )
    })

    cat("Reddit thread URLs found:", nrow(reddit_posts), "\n")

    if (nrow(reddit_posts) > 0) {
      to_fetch <- head(reddit_posts$url, reddit_max_threads)
      raw_list <- lapply(to_fetch, function(u) {
        tryCatch(
          fetch_reddit_thread(u),
          error = function(e) {
            message("Reddit content fetch failed for ", u, ": ", conditionMessage(e))
            data.frame()
          }
        )
      })

      # combine only non-empty results
      raw_df <- dplyr::bind_rows(Filter(function(x) is.data.frame(x) && nrow(x) > 0, raw_list))
    } else {
      message("No Reddit threads matched the search query.")
      raw_df <- data.frame()
    }

    # save the raw data to csv for future use and to avoid hitting api limits while developing
    write.csv(raw_df, reddit_cache, row.names = FALSE)
  }
})
```

```
## parsing URLs on page 1...
## Reddit thread URLs found: 12
```

```r
cat("Rows collected:", nrow(raw_df), "\n")
```

```
## Rows collected: 89
```

```r
cat("Posts vs comments:\n")
```

```
## Posts vs comments:
```

```r
print(table(raw_df$type))
```

```
##
## comment    post
##      77      12
```

```r
# we fetched via outscrapper which was exported as a a csv so we just read the file here :)
# due to limitations with the api price, only a subset of reviews are collected, from a subset of razr

#cat("Reviews collected:", nrow(amazon_reviews), "\n")
#cat("Rating distribution:\n")
#print(table(amazon_reviews$rating))
```

```r
fetch_youtube_comments <- function(video_id, max_comments = 200, api_key = Sys.getenv("YOUTUBE_KEY")) {
  if (identical(api_key, "")) {
    stop("YOUTUBE_KEY is not set. Load it from .env or your environment before knitting.")
  }

  all_comments <- list()
  page_token   <- NULL

  repeat {
    query <- list(
      part       = "snippet",
      videoId    = video_id,
      maxResults = 100,
      textFormat = "plainText",
      key        = api_key
    )

    # Add pagination token if we have one
    if (!is.null(page_token)) {
      query$pageToken <- page_token
    }

    resp <- httr::GET(
      "https://www.googleapis.com/youtube/v3/commentThreads",
      query = query
    )

    if (httr::http_error(resp)) {
      error_body <- tryCatch(
        httr::content(resp, as = "text", encoding = "UTF-8"),
```

```
        error = function(e) ""
      )
      message("YouTube API error: ", httr::status_code(resp), if (nzchar(error_body)) paste0(" - ", err
      break
    }

    parsed <- jsonlite::fromJSON(
      httr::content(resp, as = "text", encoding = "UTF-8"),
      flatten = TRUE
    )

    # Extract top-level comments
    items <- parsed$items

    if (is.null(items) || nrow(items) == 0) break

    batch <- data.frame(
      comment_id    = items$id,
      author        = items$snippet.topLevelComment.snippet.authorDisplayName,
      body          = items$snippet.topLevelComment.snippet.textDisplay,
      like_count    = items$snippet.topLevelComment.snippet.likeCount,
      reply_count   = items$snippet.totalReplyCount,
      date          = items$snippet.topLevelComment.snippet.publishedAt,
      stringsAsFactors = FALSE
    )

    all_comments[[length(all_comments) + 1]] <- batch
    collected <- sum(sapply(all_comments, nrow))
    message("Collected: ", collected, " comments")

    # Check for next page
    page_token <- parsed$nextPageToken

    if (is.null(page_token) || collected >= max_comments) break

    Sys.sleep(0.5)
  }

  if (length(all_comments) == 0) return(data.frame())

  dplyr::bind_rows(all_comments) %>%
    dplyr::mutate(
      source  = "youtube",
      # Like count as proxy for sentiment strength
      segment = dplyr::case_when(
        like_count >= quantile(like_count, 0.66, na.rm = TRUE) ~ "positive",
        like_count <= quantile(like_count, 0.33, na.rm = TRUE) ~ "negative",
        TRUE ~ "neutral"
      )
    )
}
razr_videos <- list(
  mkbhd      = "8om1eJrO2lU",   # MKBHD Razr 2025 review
```

```
  dave2d     = "JTT67FSc8j8",   # shortCircuit review
  mrmobile   = "YGTkjchlVJk",   # MrMobile review
  phonearena = "b3Ijuf7DHcQ"    # PhoneArena review
)

# to save on api calls, we check if we have a local csv of comments already, if not we fetch and save f
if (file.exists("data/youtube_comments.csv")) {
  youtube_all <- read.csv("data/youtube_comments.csv", stringsAsFactors = FALSE)
  cat("Loaded YouTube comments from CSV:", nrow(youtube_all), "\n")
} else {
  youtube_all <- purrr::map2_dfr(
    razr_videos,
    names(razr_videos),
  ~   fetch_youtube_comments(.x, max_comments = 200) %>%
        dplyr::mutate(channel = .y)
  )
  # Save to CSV for future runs
  write.csv(youtube_all, "data/youtube_comments.csv", row.names = FALSE);
}
```

```
## Loaded YouTube comments from CSV: 704
```

```
cat("YouTube comments collected:", nrow(youtube_all), "\n")
```

```
## YouTube comments collected: 704
```

```
print(table(youtube_all$channel))
```

```
##
##     dave2d      mkbhd   mrmobile phonearena
##        200        200        200        104
```

Pre-Processing

Side note maybe we dont have to do all of them based on the quality of the data we get, but we should at least explain how we would do them and why they are important. ## Filtering & Cleaning Explain how we cleaned the data and why.

Tokenization

Explain how we tokenized the data and why.

Stop Words Removal

Explain how we removed stop words and why.

Stemming & Lemmatization

Frequency Filter

Synonym consolidation

Summary Statistics

Topic Modeling (LDA)

Build Document

Optimal K

Fit Model

Stability

Results

Topic Word Distributions

Sometother shit

Reflection